

Ingredient Check

A browser-based food-label scanner that reads an ingredients list, explains the additives it finds, and grades the pack against the shopper's health conditions and a child's age.

This document describes the product as built: purpose, scope, architecture, matching logic, data, privacy model, and known limits. It is written so you can walk an interviewer through the work without needing the source open.

Contents

1. Purpose and the problem it solves
2. What a user actually does
3. Scope: what it is and what it is not
4. Features
5. Architecture and file map
6. Technologies
7. Ingredient matching
8. Condition engine
9. Kids safety
10. OCR and nutrition parsing
11. Reports and PDF export
12. Profiles, storage, and privacy
13. Interface and experience
14. How to run it
15. Outcomes and talking points
16. Limitations and honest answers

1. Purpose and the problem it solves

Packaged food in India and elsewhere lists ingredients for regulators, not for shoppers. Names are split (sugar, invert syrup, liquid glucose), codes appear as INS 621 or E211, and a "no added sugar" claim can sit above a list of syrups. People managing diabetes, blood pressure, PCOD, asthma, heart or kidney disease, or a sensitive gut are told to avoid families of additives, but they cannot reliably decode a pack in an aisle.

Ingredient Check turns a photo (or a paste) of the ingredients panel into a plain-language report: which additives are present, why they matter, which of the user's selected conditions they clash with, and the youngest age the product is suitable for a child. It is an educational decision aid, not a diagnosis and not a substitute for a clinician.

2. What a user actually does

The product is a two-view static site. Hash routing switches between a marketing home page ([#/](#)) and the scanner ([#/scan](#)).

On the scan screen the flow is deliberately linear:

- bullet My conditions — a native multi-select of nine health conditions. Several can be ticked at once.
- bullet Is this for a child? — optional. Kids safety already runs on every scan; the checkbox only leads the report with the age rating.
- bullet Drop the label — photograph the ingredients list, or type/paste the text if OCR will struggle.
- bullet Create a full report — a photo starts analysis as soon as it is added; the button is for pasted text or a refresh after changing conditions.
- bullet Download PDF — a keepable copy of the same verdict, findings, and condition notes.

History lives in a side drawer. Opening an old scan rebuilds the report from the stored extracted text, so conditions can be changed later and the same label re-graded.

3. Scope: what it is and what it is not

In scope

- bullet Packaged grocery labels: biscuits, namkeen, chips, drinks, sauces, instant mixes.
- bullet Text matching of common additive names, Indian INS numbers, EU E-numbers, and everyday grocery staples (maida, palmolein, jaggery, vanaspati, edible common salt).
- bullet Nine selectable adult conditions plus a always-on kids age floor.
- bullet On-device OCR, local history, and optional local profiles for a shared computer.

Out of scope

- bullet It is not medical advice, not a clinical guideline, and not 100% accurate in a medical sense. Risk depends on amount, frequency, and the person.
- bullet It does not contact a backend, does not upload photos, and does not sync across devices.
- bullet It does not claim to list every additive in existence. The databases are curated.
- bullet It does not replace a dietitian's advice for renal protein, potassium from whole foods, or insulin dosing.

4. Features

Feature	What the user gets
Label reading	Tesseract.js OCR on a contrast-boosted photo, or paste the ingredients text. Both paths produce the same report.
Additive findings	Each match has a display name, severity (high / moderate / low), what it does in food, and associated risks if eaten often.
Condition grading	Avoid vs limit groups, the exact term that hit, and a short why. Verdict is “not recommended” if any avoid group fires.
Kids safety	Every scan gets a minimum age from the strictest matching rule (honey under 1, caffeine under 12, and so on).
Nutrition flags	Sugar, fat, protein, and sodium parsed from the panel and compared with per-condition limits (e.g. high-salt at 0.6 g sodium / 100 g).

Feature	What the user gets
Product swaps	If the extracted text names a known product (Lay's, Maggi, Kurkure, cola, biscuits, namkeen, energy drinks), simpler alternatives are suggested.
PDF report	jsPDF builds an A4 file from the current verdict, findings, conditions, kids rating, and extracted text.
History and profiles	Up to 24 scans per profile in localStorage. Guest vs signed-in scopes do not mix.

Feature map as implemented in the current build.

5. Architecture and file map

There is no build step and no framework. Five JavaScript files load as classic scripts and share globals. Data files never touch the DOM; **app.js** is the only UI layer.

File	Responsibility
index.html	Home page, scanner, history drawer, profile and sign-in modals. Scripts load: ingredient-database, condition-database, profile, app, then Tesseract and jsPDF from a CDN (deferred).
styles.css	Visual system: paper cream ground, Fraunces for display type, Source Sans 3 for UI, forest green #1e5c51.
ingredient-database.js	INGREDIENT_DB (canonical additives and grocery staples with aliases, severity, risks) and PRODUCT_ALTERNATIVES (name-based swaps).
condition-database.js	CONDITION_LIBRARY, KIDS_SAFETY_RULES, shared term lists, and the matching engine (normalise, collapse INS/E codes, longest-match, negations, safe-context skips).
profile.js	Local profiles: salted password hash, session id, Auth.scope() used to namespace storage.
app.js	Routing, OCR pipeline, findIngredients, report HTML, PDF download, history, condition dropdown.
serve.py	Threaded HTTP server on 127.0.0.1 with Cache-Control: no-store so reloads pick up edits. Default port 8765.

Load order matters: databases and Auth must exist before app.js runs. CDN libraries are deferred so a blocked network does not prevent the paste-text path from working.

6. Technologies

bullet HTML, CSS, vanilla JavaScript — no React, no bundler, no package.json. Easy to explain and to run.

bullet Tesseract.js 5 (jsDelivr) — client-side OCR, English traineddata, runs in the browser.

bullet jsPDF 2.5.1 — client-side PDF of the current report.

bullet localStorage — conditions, history, and profiles, namespaced per Auth.scope().

bullet Web Crypto — SHA-256 of salt + password when crypto.subtle is available (HTTPS or localhost).

 Fallback hash on plain http:// file origins.

bullet Canvas / createImageBitmap — upscale and contrast-boost the photo before OCR.

bullet Python 3 http.server (custom threaded, no-cache handler) — local development; camera capture needs localhost or HTTPS.

bullet Google Fonts — Fraunces and Source Sans 3. The app still works if fonts fail to load.

If an interviewer asks “why no backend?”: the photos never need to leave the device, there is no user account server, and the matching data is small enough to ship as JS. The trade-off is no cross-device sync and a database that updates only when the files do.

7. Ingredient matching

`findIngredients()` in `app.js` is the general additive finder. It is separate from the condition engine so a guest with no conditions still sees what is in the pack.

Pipeline

- bullet** Prefer the ingredients block if OCR captured the whole reverse of the pack (cut at “Nutritional Information”, “Manufactured by”, FSSAI licence, and similar).
- bullet** `enhanceOcrText()` folds INS 621 / E-621 / OCR “INS” into e621, and repairs common misreads (malda → maida, tartrazin, vanaspati, carageenan).
- bullet** Each `INGREDIENT_DB` item tries aliases longest-first. Short codes (msg, e102, bha) use word-boundary regex so they do not match inside longer words.
- bullet** Overlapping spans: the longer match wins, so “high fructose corn syrup” does not also report “corn syrup”.
- bullet** Bare “sugar” is handled with a word-boundary helper that ignores “sugar-free” and nutrition-table “of which sugars”.

What the database contains

About 61 canonical entries covering sweeteners, preservatives, colours, fats, flavour enhancers, emulsifiers, phosphates, grocery staples (maida, salt, lecithin, milk solids, sorbitol), and a handful of higher-concern additives (titanium dioxide, brominated vegetable oil, potassium bromate). Each entry has aliases, a category, severity, a short note, risk bullets, and a “watch for” line. Product swaps are a second list keyed by brand/product aliases (Lay’s Magic Masala, Maggi, Kurkure, Coca-Cola, biscuits, namkeen, energy drinks).

8. Condition engine

`condition-database.js` owns both the data and the matcher. `evaluateConditions(labelText, ids)` returns only conditions that actually hit, sorted with “avoid” verdicts first.

Shared term arrays (sugars, refined starches, trans fats, sodium, flavour enhancers, sulphites, benzoates, polyols, phosphates, and so on) are reused across conditions so the same pack wording is not maintained in nine places. Each group has a level of avoid or limit, a title, a why, and optional negations.

Condition	Avoid (typical)	Limit (typical)
Diabetes Type A	Added sugars and syrups; trans / hydrogenated fats	Artificial sweeteners
Diabetes Type B	Those sugars plus maida, maltodextrin, modified starch; trans fats	Artificial sweeteners
High BP	Salt, soy sauce, MSG, cured-meat nitrites, liquorice	Baking powder and other sodium additives, caffeine
Low BP	Alcohol, diuretic extracts	Large refined-carb loads, heavy caffeine. Salt is not flagged.

Condition	Avoid (typical)	Limit (typical)
Heart disease	Trans fats, salt, MSG, nitrite-cured meats	Palmolein / coconut / ghee, added sugars, caffeine
Kidney disease	Added phosphates (INS 338–452), salt, MSG, potassium chloride / low-sodium salt, aluminium anti-caking agents	Cured meats, caffeine
PCOD	Sugars, maida, trans fats	Palmolein (not treated as trans fat), colours, MSG, caffeine
Asthma	Sulphites, azo dyes (tartrazine, sunset yellow), benzoates	MSG, TBHQ/BHA, unnamed “class II preservative”, carmine / annatto
Sensitive gut	Carrageenan, polysorbates, polyols	Inulin / FOS, onion and garlic powder, milk solids. Soya lecithin is not treated as a gut-avoid emulsifier.

Condition groups as implemented. Wording on the pack is matched, not clinical staging of the disease.

Matching details worth defending in an interview

- Longest term wins inside a group, so nested names do not double-count.
- INS / E collapse — “INS 621”, “E-621”, and OCR “INS 621” all become e621 before matching.
- Negations — “no added sugar”, “sugar-free”, “low salt” prefixes do not fire the sugar or salt groups.
- Safe context — “whole wheat flour” is not maida; “non-hydrogenated” is not trans fat; “wine vinegar” is not alcohol; “peanut butter” is not dairy butter; “cream of tartar” is not cream.
- Ingredients vs nutrition table — if both are in the OCR dump, matching uses the ingredients list so “of which sugars 1.2 g” does not flag diabetes on a whole-wheat loaf.
- Low BP inverts high BP — sodium warnings are omitted and the report states that explicitly.
- Palmolein vs trans fat — palmolein is a saturated tropical oil (heart/PCOD limit), not hydrogenated fat.

9. Kids safety

KIDS_SAFETY_RULES runs on every scan, whether or not the pack is marketed to children. Each rule has a minAge and severity. The product floor is the maximum minAge among hits. Copy such as “for kids” or “tiffin” only changes whether the UI treats it as a children’s product; it does not turn the rules off.

Floor	Triggered by
Under 1	Honey (infant botulism risk)
Under 5	Added sugar, artificial sweeteners, high-sodium additives, sugar alcohols (polyols)
Under 10	Artificial colours, benzoates, trans fats, nitrite-cured meat, synthetic antioxidants (BHA, TBHQ)
Under 12	Caffeine (coffee extract, guarana, energy blends — not cocoa solids on a biscuit)

Age floors. The on-screen rating uses the strictest match.

10. OCR and nutrition parsing

Photo path

- bullet User drops or chooses an image (camera capture is offered on phones).
- bullet `prepareImageForOcr()` scales up to about 2000 px on the long side and boosts contrast (darken below a grey threshold, lift highlights) so small pack print is easier for Tesseract.
- bullet `Tesseract.recognize(blob, "eng")` returns text. Progress is shown as a percentage.
- bullet If the library failed to load, the UI asks the user to type the ingredients instead — same matcher.

Nutrition

`parseNutritionText()` looks for sugar, fat, protein, and sodium (mg converted to grams). Salt grams are converted to sodium by dividing by 2.5 when sodium is missing. Per-condition nutrition[] limits then flag “high for heart disease” and similar. Interviewers sometimes ask if we trust the panel over the ingredients list: ingredients drive additive matching; the panel only adds quantitative flags when numbers are present.

11. Reports and PDF export

`displayFindings()` builds the on-screen report: a verdict, counts of high/moderate/low findings, per-condition avoid/limit groups with the terms found, kids rating, nutrition, additive cards, optional swaps, the extracted text in a disclosure, and Download PDF.

Verdict logic: if any selected condition has an avoid-level hit, the headline is “Not recommended for you” and lists those conditions. If only limit-level hits, the tone is caution. If nothing matches the selected conditions, the report still shows general additive findings.

jsPDF writes an A4 document from `lastReport` (verdict, findings, condition groups, kids, nutrition, extracted text). If the CDN script did not load, the user is told to check the connection rather than failing silently.

12. Profiles, storage, and privacy

There is no server-side account. A “profile” is a `localStorage` record so two people on one laptop do not share conditions and history. The UI is explicit that this is not real authentication: anyone with the browser can read the data.

- bullet Users key: `ingredient_check_users` — id, name, email, salt, hash, `createdAt`.
- bullet Session: `ingredient_check_session` — the current user id, or guest.
- bullet Scoped keys: `ingredient_check_conditions:u_`, `ingredient_check_history:u_`, and the guest equivalents.
- bullet Passwords: 16-byte salt + SHA-256 when `SubtleCrypto` exists; a weaker local hash if the page is opened as a file on http that is not localhost.
- bullet History cap: 24 scans. Extracted text is truncated when stored.
- bullet Photos are never stored; they exist only as a blob URL for preview and OCR, then are discarded.
- bullet Deleting a profile removes that user’s stored records. Clearing site data wipes everything.

If asked “is this GDPR / production-ready auth?”: no. It is a convenience namespace on a trusted device. Production would need a real backend, proper password hashing (Argon2/bcrypt on a server), and a privacy policy. That honesty is better in an interview than overselling `localStorage`.

13. Interface and experience

The visual system is editorial rather than generic SaaS: warm paper background, serif headlines, forest green, hairline rules instead of heavy card shadows. The scan screen is a four-step worksheet. The conditions control is a native `<details>` / `<summary>` menu so it opens even if JavaScript on the page is delayed. Checkboxes are also in the HTML as a fallback before `app.js` re-renders them from `CONDITION_LIBRARY`.

Responsive behaviour: at about 720 px the history tab becomes a bottom-right chip; nav section links hide on very small screens; the hero stacks. Camera capture is offered via the file input's capture attribute on phones.

14. How to run it

From the project folder:

python3 serve.py 8765 then open **http://127.0.0.1:8765/**

Opening index.html as a file works for paste-text, but the camera and some crypto APIs expect localhost or HTTPS. The custom server is threaded and sends no-store cache headers so CSS and JS edits appear on a normal reload.

Best scan results: crop to the ingredients list, even light, no plastic glare. If OCR is messy, paste the list — the report path is identical.

15. Outcomes and talking points

What you can claim, precisely:

- bullet** An end-to-end product: capture → OCR → domain matching → personalised report → PDF, with no backend.
- bullet** Domain modelling: nine conditions as data, not hard-coded if/else per disease, so adding a condition is a data change.
- bullet** Practical NLP-lite on dirty OCR: code collapsing, negation, longest-match, and Indian pack vocabulary (maida, palmolein, vanaspati, INS numbers).
- bullet** Product judgement: educational disclaimer, kids rules always on, low BP not inheriting salt warnings, palmolein not labelled as trans fat.
- bullet** Privacy stance: photos stay on device; storage is local and scoped.
- bullet** Front-end craft: routing, empty/error/OCR-failure states, history replay, accessible native controls.

Sample walkthrough for a demo

Tick Diabetes Type B and Asthma. Paste: refined wheat flour (maida), sugar, palmolein, invert syrup, sodium metabisulphite, tartrazine, MSG, sodium benzoate, maltodextrin, salt. Expect a “not recommended” verdict for both conditions, maida and sugars on Type B, sulphites/dye/benzoate on asthma, and a kids floor driven by colours (10+) or caffeine if present (12+).

16. Limitations and honest answers

Likely question	Straight answer
Is the report 100% accurate?	No. It is as accurate as the photo, the OCR, and a curated term list. Unusual spellings and unnamed “class II preservative” without a number are ambiguous. It is educational, not a lab assay.
Why vanilla JS?	The app is a few screens and static data. A framework would add a build step without helping OCR or matching. The matching engine is easier to test as plain functions.
How do you test matching?	The matchers are pure functions on strings. Fixtures include biscuits, chips, cola, sugar-free polyol packs, whole-wheat bread, wine vinegar, and nutrition-table traps.

Likely question	Straight answer
Security of profiles?	Local convenience only. Hashes are not a substitute for server-side auth. XSS on the origin could read localStorage.
What would you build next?	A maintained additive taxonomy, better OCR (region detect the ingredients block), i18n for Hindi packs, and an optional signed-in cloud backup — with a real privacy design.
Why Tesseract in the browser?	Keeps photos on the device and works offline after the first script load. Quality is weaker than a cloud vision API on curved glossy packs; that is why paste exists.

Closing summary

Ingredient Check is a complete, client-side product that reads a grocery label and explains it in the language of the shopper's conditions and of kids' safety. The interesting engineering is not the framework choice; it is the matching layer on messy real-world pack text, the separation of data from UI, and the decision to keep personal health context on the device. Treat the output as a well-informed second look at the back of the pack, then defer to a clinician for anything that changes treatment.

Document generated from the Ingredient Check codebase. Educational information only.